

Arquitectura Empresarial E-Commerce

Este documento define la estructura técnica y el código funcional para una plataforma de comercio electrónico de clase empresarial. La solución integra un frontend moderno en Angular 18, microservicios en .NET 9, mensajería asíncrona con RabbitMQ y automatización de infraestructura mediante Terraform, Ansible y Kubernetes.

Estructura del Proyecto

La organización del repositorio sigue un patrón de monorepo para facilitar la orquestación de contenedores y el despliegue continuo.

- **/src/client:** Aplicación Frontend en Angular 18+.
- **/src/services/Catalog.API:** Microservicio de catálogo en .NET 9.
- **/src/workers/ServiceNow.Worker:** Worker en .NET 9 para integración con ServiceNow.
- **/infrastructure/terraform:** Scripts de Python/HCL para aprovisionamiento.
- **/infrastructure/ansible:** Playbooks para configuración de servidores NGINX.
- **/infrastructure/k8s:** Manifiestos de Kubernetes.
- **/deployment:** Dockerfiles y archivos de composición.

Capa de Frontend: Angular 18+

Utilizamos componentes Standalone y Signals para una gestión de estado eficiente.

Componente de Catálogo (catalog.component.ts)

```
import { Component, OnInit, signal } from '@angular/core';
```

```
import { CommonModule } from '@angular/common';
```

```
import { HttpClient } from '@angular/common/http';
```

```
interface Product {
```

```
  id: number;
```

```
  name: string;
```

```
  price: number;
```

```
}
```

```
@Component({
```

```
  selector: 'app-catalog',
```

```
  standalone: true,
```

```
  imports: [CommonModule],
```

```
  template: `
```

```
    <div class="catalog-container">
```

```
      <h1>Catálogo de Productos</h1>
```

```
      <div class="grid">
```

```
        @for (product of products(); track product.id) {
```

```
          <div class="card">
```

```
            <h3>{{ product.name }}</h3>
```

```
            <p>{{ product.price | currency }}</p>
```

```
            <button (click)="createIncident(product)">Reportar Problema</button>
```

```
          </div>
```

```
        }
```

```

        </div>

    </div>

    ,

})

export class CatalogComponent implements OnInit {

    products = signal<Product[]>([]);

    constructor(private http: HttpClient) {}

    ngOnInit() {

        this.http.get<Product[]>('/api/catalog').subscribe(data => {

            this.products.set(data);

        });

    }

    createIncident(product: Product) {

        this.http.post('/api/catalog/report', { productId: product.id })

            .subscribe(() => alert('Incidente enviado a ServiceNow'));

    }

}

```

Capa de Backend: .NET 9 ASP.NET Core

El API de Catálogo procesa las solicitudes y publica eventos en RabbitMQ.

Catalog Controller (Program.cs)

```
using Microsoft.AspNetCore.Mvc;

using RabbitMQ.Client;

using System.Text;

using System.Text.Json;

var builder = WebApplication.CreateBuilder(args);

var app = builder.Build();

app.MapGet("/api/catalog", () => new[]

{

    new { Id = 1, Name = "Laptop Enterprise", Price = 1200.00 },

    new { Id = 2, Name = "Monitor 4K", Price = 450.00 }

});

app.MapPost("/api/catalog/report", async ([FromBody] ReportRequest request) =>

{

    var factory = new ConnectionFactory { HostName = "rabbitmq-service" };

    using var connection = await factory.CreateConnectionAsync();

    using var channel = await connection.CreateChannelAsync();

    await channel.QueueDeclareAsync(queue: "servicenow_tasks", durable: true,
exclusive: false, autoDelete: false);

    var message = JsonSerializer.Serialize(request);

    var body = Encoding.UTF8.GetBytes(message);
```

```
await channel.BasicPublishAsync(exchange: string.Empty, routingKey:
"servicenow_tasks", body: body);
```

```
return Results.Accepted();
```

```
});
```

```
app.Run();
```

```
public record ReportRequest(int ProductId);
```

Integración con ServiceNow: Worker Service

Este servicio actúa como consumidor de RabbitMQ y crea tickets en ServiceNow.

Worker.cs

```
p
public class ServiceNowWorker : BackgroundService
{
    private readonly ILogger _logger;
    private readonly HttpClient _httpClient;public
    ServiceNowWorker(ILogger<ServiceNowWorker> logger, IHttpClientFactory
    httpClientFactory)

    {

        _logger = logger;

        _httpClient = httpClientFactory.CreateClient("ServiceNowApi");

    }

    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
```

```

{

    var factory = new ConnectionFactory { HostName = "rabbitmq-service" };

    using var connection = await factory.CreateConnectionAsync();

    using var channel = await connection.CreateChannelAsync();

    var consumer = new AsyncEventingBasicConsumer(channel);

    consumer.ReceivedAsync += async (model, ea) =>
    {

        var body = ea.Body.ToArray();

        var message = Encoding.UTF8.GetString(body);

        _logger.LogInformation("Procesando ticket para ServiceNow: {Message}",
            message);

        // Lógica de integración con API de ServiceNow

        var response = await
            _httpClient.PostAsJsonAsync("https://dev-instance.service-now.com/api/now/table/
            incident", new {

                short_description = $"Error reportado en producto: {message}",

                severity = "2"

            });

    };
};

```

```
    await channel.BasicConsumeAsync(queue: "servicenow_tasks", autoAck: true,
consumer: consumer);
```

```
    await Task.Delay(Timeout.Infinite, stoppingToken)
;
}
}
```

Infraestructura como Código (IaC)

Terraform mediante Python (CDKTF Style)

Utilizamos el patrón de generación de recursos mediante scripts de Python para mayor flexibilidad en entornos dinámicos.

```
```python
```

```
import os
```

```
from cdktf import App, TerraformStack
```

```
from cdktf_cdktf_provider_aws.provider import AwsProvider
```

```
from cdktf_cdktf_provider_aws.instance import Instance
```

```
class EnterpriseStack(TerraformStack):
```

```
 def __init__(self, scope, id):
```

```
 super().__init__(scope, id)
```

```
 AwsProvider(self, "AWS", region="us-east-1")
```

```
 Instance(self, "K8sNode",
```

```
 ami="ami-0c55b159cbfafa1f0",
```

```
 instance_type="t3.medium",
```

```
 tags={"Name": "Ecom-Production-Node"}

)

app = App()

EnterpriseStack(app, "ecom-enterprise")

app.synth()
```

## Orquestación: Kubernetes Manifests

apiVersion: apps/v1

kind: Deployment

metadata:

name: catalog-api

spec:

replicas: 3

selector:

matchLabels:

app: catalog-api

template:

metadata:

labels:

app: catalog-api

spec:

containers:

- name: catalog-api

image: enterprise-registry.com/catalog-api:v1.0.0

ports:

- containerPort: 80

env:

- name: RabbitMQ\_\_Host

value: "rabbitmq-service"

---

apiVersion: v1

kind: Service

metadata:

name: catalog-service

spec:

selector:

app: catalog-api

ports:

- protocol: TCP

port: 80

targetPort: 80

# Automatización con Ansible

Playbook para configurar el servidor de entrada NGINX que actúa como Gateway.

## site.yml (Jinja2 Template)

|

- name: Configurar Gateway NGINX  
hosts: gateways  
become: yes  
tasks:
  - name: Instalar NGINX  
apt:  
name: nginx  
state: present
  - name: Aplicar configuración de balanceo  
template:  
src: templates/nginx.conf.j2  
dest: /etc/nginx/nginx.conf  
notify: Reiniciar NGINX

handlers

:

- name: Reiniciar NGINX

service:

name: nginx

state: restarted

### nginx.conf.j2

```nginx

http {

upstream backend_cluster {

server :8080;

}

```
server {  
  
    listen 80;  
  
    location / {  
  
        proxy_pass http://backend_cluster;  
  
        proxy_set_header Host $host;  
  
    }  
  
}  
  
}
```

Dockerización de Producción

Dockerfile (.NET 9)

FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build

WORKDIR /app

COPY *.csproj ./

RUN dotnet restore

COPY . ./

RUN dotnet publish -c Release -o out

FROM mcr.microsoft.com/dotnet/aspnet:9.0

WORKDIR /app

COPY --from=build /app/out .

ENTRYPOINT ["dotnet", "Catalog.API.dll"]

Datos de Contacto y Seguimiento

Para la ejecución de este despliegue, se requiere la aprobación de los responsables de infraestructura.

| Rol | Responsable | Fecha de Aprobación |
|------------------|--|--|
| Cloud Architect |  Person |  Date |
| DevOps Lead |  Person |  Date |
| Security Officer |  Person |  Date |